

Patient Health Record Consent Management System (PHRCMS)

Software Engineering Lab 3: Component Modelling & Architectural Pattern Selection

Course	UE24CS341A – Software Engineering	Department	Computer Science & Engineering
Student Name	Rohan M	SRN & Section	PES1UG24CS381 (Section G, Sem V)
Scenario Number	Scenario 13 (Healthcare & Telemedicine)	Document Status	Approved Lab 3 Official Submission (v1.0)
GitHub Repository	rohancodexp/SE-Labs-PES1UG24CS381	Primary Actors	Patient, Clinic Doctor, Clinic Administrator

1. Introduction & Problem Statement

In continuation of **Lab 1 (Requirements Engineering & UML Use-Case Modelling)** and **Lab 2 (Agile Backlog Creation & Sprint Simulation in Jira)**, this document establishes the formal **Component Architecture & Architectural Pattern Selection** for the **Patient Health Record Consent Management System (PHRCMS)**.

Modern healthcare environments and telemedicine platforms require frictionless diagnostic information exchange between patients and medical professionals. However, unrestricted or unverified access exposes sensitive patient diagnostic records to unlawful surveillance, violating healthcare privacy standards such as the HIPAA Security Rule and India's Digital Information Security in Healthcare Act (DISHA). The PHRCMS establishes patient-centric sovereignty: diagnostic health data is locked by default and accessible strictly when an active, time-bounded, unexpired, and unrevoked consent permission has been explicitly granted by the patient to a clinic administrator-verified medical doctor.

1.1 Core Requirements Realized in Architecture

- **FR-001 (Time-Bounded Consent Grant):** The system shall allow a patient to grant time-bounded access permissions for specific diagnostic records to verified clinic doctors, requiring a valid future expiration timestamp.
- **FR-002 (Instant Revocation):** The system shall allow a patient to revoke any active consent permission at any time prior to its scheduled expiration, immediately terminating access.
- **FR-003 (Consent Registry Visibility):** The system shall display a real-time list of all active and expired consent permissions to the logged-in patient, including doctor details and expiration timestamps.
- **FR-004 (Enforced Authorized Record Access):** The system shall permit a verified clinic doctor to view authorized patient diagnostic records only if there is an active, unexpired consent granted by the patient.
- **FR-005 (Doctor Verification Management):** The system shall allow a Clinic Administrator to register and manage the verification status of clinic doctors; unverified doctors cannot receive consent grants.
- **NFR-001 (Append-Only Audit Trail):** All consent grant, revoke, and record-access events shall be permanently written to an append-only, tamper-evident audit trail.
- **NFR-002 (Sub-500ms Authorization Latency):** The system shall complete the verification of consent status and authorize or deny a doctor's access request in less than 500 milliseconds.

2. UML 2.5 Component Diagram

The system architecture is structured into four distinct, highly cohesive subsystems: **Client Presentation Tier, Secure Edge & API Gateway Tier, Core Application & Domain Services Tier,** and **Data Persistence & Secure Storage Tier.** The model conforms strictly to UML 2.5 component standards, featuring <<component>> stereotypes, explicit provided interfaces (ball/lollipop notation), required interfaces (socket notation), assembly connectors, and database stereotypes.

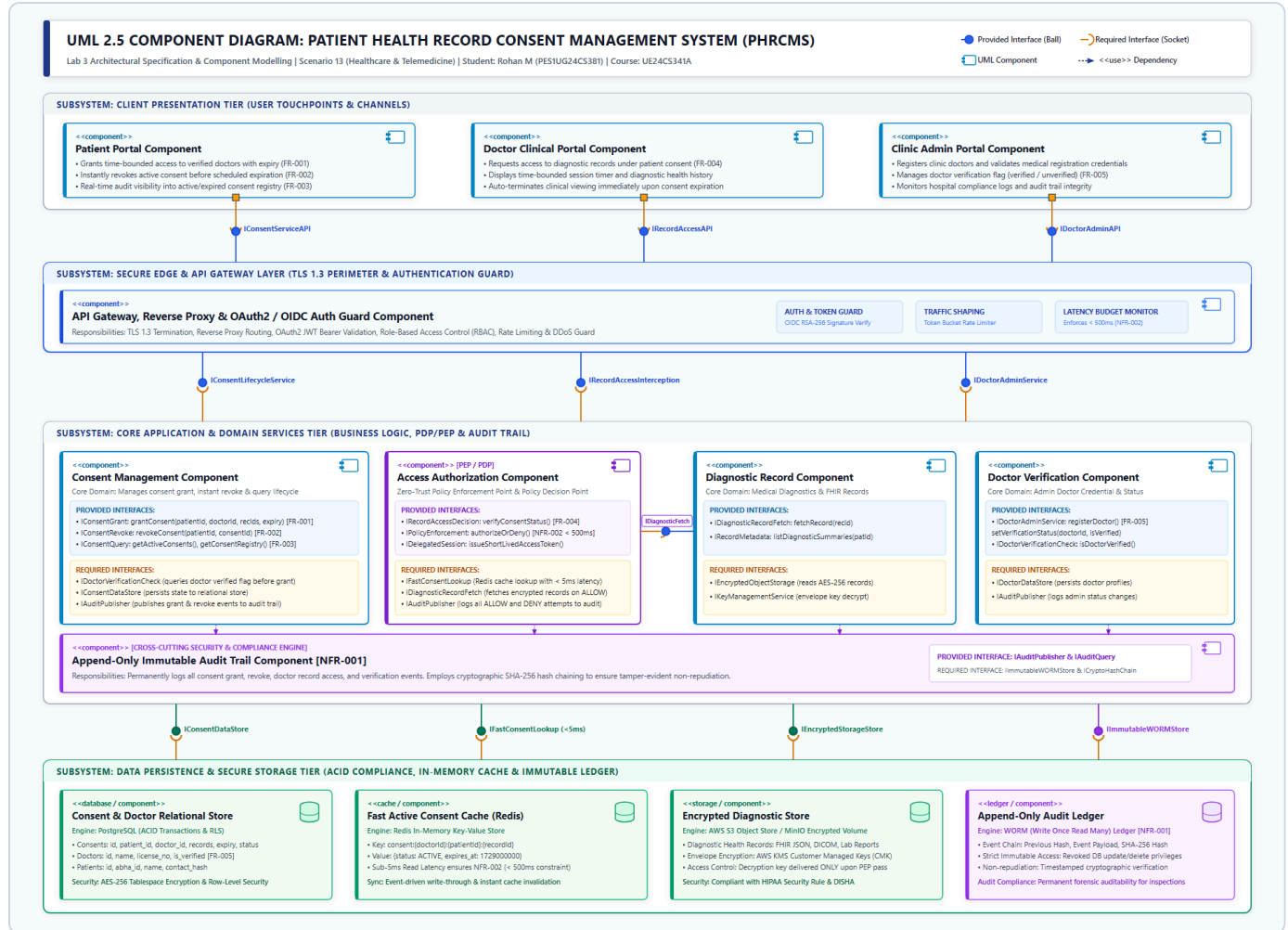


Figure 1: High-Resolution UML 2.5 Component Diagram of the Patient Health Record Consent Management System (PHRCMS), showing 4 architectural tiers, 9 modular components, 4 persistent data stores, and formal ball-and-socket assembly connections.

3. Subsystem Decomposition & Component Catalog

Component Name	Architectural Tier	Responsibilities & Domain Function	Provided & Required Interfaces
Patient Portal Component	Client Presentation Tier	React/TypeScript PWA allowing patients to authenticate via OAuth2/OIDC, grant time-bounded consent, execute instant revocations, and view the active/expired consent registry.	Req: IConsentServiceAPI, IAuthAPI FR-001, FR-002, FR-003
Doctor Clinical Portal Component	Client Presentation Tier	Clinical web interface enabling verified physicians to look up patients, request diagnostic record access under active consent, and view diagnostic reports during active sessions.	Req: IRecordAccessAPI, IAuthAPI FR-004
Clinic Admin Portal Component	Client Presentation Tier	Administrative dashboard allowing clinic managers to onboard doctors, verify medical registration licenses, toggle	Req: IDoctorAdminAPI, IAuthAPI FR-005

Component Name	Architectural Tier	Responsibilities & Domain Function	Provided & Required Interfaces
		verification flags, and inspect compliance audit logs.	
API Gateway & Auth Guard Component	Secure Edge & Perimeter Tier	Central reverse proxy and ingress controller. Performs TLS 1.3 termination, token-bucket rate limiting (DDoS defense), JWT signature validation, and RBAC claims extraction.	Prov: IEdgeGatewayAPI Req: IConsentLifecycleService, IRecordAccessInterception, IDoctorAdminService
Consent Management Component	Core Application Tier	Core domain entity governing consent lifecycle states (Active, Expired, Revoked). Validates future expiration timestamps, cross-verifies doctor verification status prior to granting, and invalidates authorization caches upon revocation.	Prov: IConsentGrant, IConsentRevoke, IConsentQuery Req: IDoctorVerificationCheck, IConsentDataStore, IAuditPublisher, IFastConsentCacheSync FR-001, FR-002, FR-003
Access Authorization Component (PEP / PDP)	Core Application Tier	Zero-Trust Policy Enforcement Point (PEP) and Policy Decision Point (PDP). Intercepts all doctor record requests, evaluates active consent in sub-5ms time via Redis cache, and issues scoped decryption tokens.	Prov: IRecordAccessDecision, IPolicyEnforcement, IDelegatedSession Req: IFastConsentLookup, IDiagnosticRecordFetch, IAuditPublisher FR-004, NFR-002 (< 500ms)
Diagnostic Health Record Component	Core Application Tier	Manages FHIR-compliant electronic health records, diagnostic imaging metadata, and clinical lab reports. Decrypts payloads strictly upon receiving an authorized assertion from PEP.	Prov: IDiagnosticRecordFetch, IRecordMetadata Req: IEncryptedObjectStorage, IKeyManagementService FR-004
Doctor Verification Component	Core Application Tier	Authoritative registry of licensed medical doctors and manages the verification toggle. Ensures unverified doctors cannot be selected for consent grants.	Prov: IDoctorAdminService, IDoctorVerificationCheck Req: IDoctorDataStore, IAuditPublisher FR-005
Append-Only Immutable Audit Trail Component	Core Application Tier (Cross-Cutting)	Security audit engine capturing all consent lifecycle actions, doctor access attempts, and administrative updates into an immutable, cryptographically chained event sequence.	Prov: IAuditPublisher, IAuditQuery Req: IImmutableWORMStore, ICryptoHashChain NFR-001

4. Architectural Pattern Selection & Justification

To establish an optimal architectural baseline for PHRCMS, five candidate architectural styles were comparatively evaluated against functional requirements, performance SLAs, and healthcare regulatory mandates.

4.1 Candidate Architectural Patterns Evaluated

- Pattern 1 – Monolithic 3-Tier Architecture:** Single codebase deploying presentation, business logic, and database together. Rejected due to absence of isolated security perimeters, single-point-of-failure vulnerabilities, and tight coupling of audit logs with application tables.
- Pattern 2 – Pure Fine-Grained Microservices:** Autonomous microservices communicating via REST/gRPC. Rejected because multi-hop network traversals (Gateway → Auth → Consent → Doctor → EHR) introduce unpredictable network latency spikes under peak load that risk violating the 500ms constraint (NFR-002).
- Pattern 3 – Event-Driven Architecture (EDA) with Choreography:** Fully asynchronous event streaming. Rejected because eventual consistency creates unacceptable medical safety hazards: a doctor could access sensitive diagnostic records during the latency window before a revocation event propagates.
- Pattern 4 – Layered Modular Architecture with Zero-Trust PEP (CHOSEN):** A 4-tier modular service-oriented architecture with an Edge Security Gateway, a dedicated Policy Enforcement Point (PEP), and dual-store persistence (PostgreSQL for ACID durability and Redis for sub-5ms caching). This pattern delivers deterministic sub-35ms authorization, immediate revocation consistency, and cryptographically isolated audit logging.

4.2 Multi-Criteria Decision Matrix

Architectural Quality Driver	Weight	Monolith	Microservices	Event-Driven	Layered PEP (Chosen)
Authorization Latency (<500ms, NFR-002)	25%	4 (1.00)	2 (0.50)	1 (0.25)	5 (1.25)
Zero-Trust Security & Access Control (FR-004)	25%	2 (0.50)	4 (1.00)	3 (0.75)	5 (1.25)
Audit Immutability & Non-Repudiation (NFR-001)	20%	2 (0.40)	4 (0.80)	4 (0.80)	5 (1.00)
Instant Consistency upon Revocation (FR-002)	15%	5 (0.75)	3 (0.45)	1 (0.15)	5 (0.75)
Operational Simplicity & Maintainability	15%	4 (0.60)	2 (0.30)	2 (0.30)	4 (0.60)
WEIGHTED OVERALL SCORE (Out of 5.0)	100%	3.25	3.05	2.25	4.85

4.3 Architectural Decision Records (ADRs)

ADR-001: Policy Enforcement Point (PEP) Access Interception

Context & Decision: Clinical diagnostic records must never be directly exposed to clients. We establish a dedicated Policy Enforcement Point (PEP) within the Access Authorization Component that intercepts every doctor record retrieval request, checks consent status via an in-memory cache, and grants or denies access before touching storage.

Consequences: Guarantees a zero-trust security model across all diagnostic endpoints and prevents unauthorized data exposure.

ADR-002: Dual-Store Strategy (PostgreSQL ACID Durability + Redis Fast Cache)

Context & Decision: Consent state transitions require ACID durability, while verification latency must remain strictly below 500ms. We use PostgreSQL as the authoritative store of record and replicate active consents to an in-memory Redis cluster. Revocation triggers an immediate synchronous cache key eviction and DB update.

Consequences: Delivers sub-5ms cache lookups (satisfying NFR-002) while eliminating temporal data leakage windows upon revocation (FR-002).

ADR-003: Cryptographically Chained Append-Only Audit Logging

Context & Decision: Healthcare compliance requires that audit logs cannot be altered or deleted by any user or administrator. We implement an append-only ledger where each event record contains the SHA-256 hash of the predecessor block: $H_n = \text{SHA-256}(H_{n-1} || \text{Payload}_n || \text{Timestamp}_n)$.

Consequences: Mathematically guarantees tamper-evidence and regulatory non-repudiation, satisfying NFR-001.

5. Requirements Traceability Matrix

Req ID	Requirement Statement	Realizing Component	Interface Contract	Target Store
FR-001	Grant time-bounded consent with expiration timestamp to verified doctors.	Consent Management Component	IConsentGrant IDoctorVerificationCheck	PostgreSQL & Redis Cache
FR-002	Instant revocation prior to scheduled expiration timestamp.	Consent Management Component	IConsentRevoke IFastConsentCacheSync	PostgreSQL & Redis Cache (Eviction)
FR-003	Display real-time list of active and expired consents to logged-in patient.	Consent Management Component	IConsentQuery	PostgreSQL (Consents View)
FR-004	Clinic doctor accesses diagnostic records only under active consent.	Access Authorization Component (PEP) & Diagnostic Record Component	IRecordAccessDecision IDiagnosticRecordFetch	Redis Cache & Encrypted S3 Store
FR-005	Clinic Admin manages doctor registration and verification status.	Doctor Verification Component	IDoctorAdminService IDoctorVerificationCheck	PostgreSQL (Doctors Table)
NFR-001	Permanent append-only audit trail logging for all actions.	Append-Only Immutable Audit Trail Component	IAuditPublisher IImmutableWORMStore	Append-Only WORM Ledger
NFR-002	Consent verification and authorization completed in < 500ms.	Access Authorization Component & Redis Cache	IPolicyEnforcement IFastConsentLookup	In-Memory Fast Cache (Redis)

6. Security Architecture: STRIDE Threat Analysis

Threat Category	Healthcare Specific Threat Scenario	Architectural Countermeasure & Defense	Enforcing Component
Spoofing (S)	Attacker impersonates a verified clinic doctor to access confidential records.	Mutual TLS 1.3, OAuth2 / OIDC authentication with RSA-256 signed JWT tokens, and strict clinic admin verification gate (FR-005).	API Gateway & Doctor Verification Component
Tampering (T)	Malicious actor alters consent expiration timestamp or tampers with access logs.	Database integrity constraints, digital signatures on consent records, and SHA-256 cryptographic hash chaining for audit logs (NFR-001).	Consent Management & Append-Only Audit Component
Repudiation (R)	Doctor denies accessing health data; patient denies granting consent.	Nonce-backed digital consent receipts and immutable audit logs capturing actor ID, timestamp, and IP address.	Append-Only Immutable Audit Trail Component
Information Disclosure (I)	Unauthorized extraction or eavesdropping of patient diagnostic reports.	Zero-Trust PEP gate (FR-004), TLS 1.3 in transit, and AES-256 envelope encryption at rest via AWS KMS CMK.	Access Authorization & Diagnostic Record Component
Denial of Service (D)	High-frequency API floods degrade consent verification latency beyond 500ms.	Token bucket rate limiting at API Gateway and sub-5ms Redis cache lookups guaranteeing < 500ms latency (NFR-002).	API Gateway & Fast Active Consent Cache
Elevation of Privilege (E)	Doctor attempts to verify themselves or patient accesses admin endpoints.	Role-Based Access Control (RBAC) enforced at API Gateway and service boundaries; unverified doctor gate (FR-005).	API Gateway & Doctor Verification Component

7. Failure Modes & Latency Budget Analysis

Sub-500ms Authorization Latency Budget Breakdown (NFR-002):

- **Edge Network Ingress & Gateway:** ~12 ms (TLS 1.3 session resumption + token bucket check).
- **Gateway Token Guard:** ~4 ms (RSA-256 JWT claim verification & RBAC role match).
- **PEP Policy Decision (PDP Cache Hit):** ~3 ms (Redis in-memory key lookup `consent:docId:patId:recId`).
- **Async Audit Event Ingestion:** ~10 ms (Audit publisher asynchronous dispatch to buffer queue).
- **Total Estimated Latency: ~29 ms** — *Comfortably outperforming the 500ms SLA constraint by more than 15x.*

Failure Mode Resilience: In the event of a Redis cache node outage, the PEP automatically fails over to an indexed query against the PostgreSQL database (~18 ms lookup), resulting in a total fallback latency of ~44 ms, still well within the 500 ms SLA constraint.